

TEMPLATE PARA ENTREGA DO PROJETO DA DISCIPLINA

DevOPs na Prática

Fase 1: Configuração e Automação Inicial

NOME DO ESTUDANTE: Samuel Aiedo

PROJETO: MedQR - Cylind Pipeline (Github Actions) & IaC (AWS CloudFormation) & Terraform

NOME DO ESTUDANTE:

SEÇÃO 1

Documentação de Planejamento

a) **DESCRIÇÃO DO PROJETO, SEUS OBJETIVOS E REQUISITOS:**

- **DESCRIÇÃO DO PROJETO:**
O MedQR é uma solução de compartilhamento seguro de fichas médicas de emergência para trabalhadores acidentados em ambiente laboral. A aplicação gera QR Codes para crachás corporativos protegidos por senha de acesso público.
- **OBJETIVOS DE DEVOPS:**
 1. Estabelecer um pipeline de Integração Contínua (CI) automatizado para garantir a qualidade, sintaxe e build do código.
 2. Implementar a Infraestrutura como Código (IaC) via Terraform para provisionar o ambiente na nuvem AWS de forma declarativa e reproduzível.
- **REQUISITOS:**
 - Repositório Git/GitHub com branching model estruturado (main e develop).
 - Execução automatizada de linters e testes unitários a cada push/pull request.
 - Isolamento de banco de dados em subnets privadas sem acesso público direto.

b) **PLANO DE INTEGRAÇÃO CONTÍNUA:**

- ESTRATÉGIA DE BRANCHING & TRIGGERS:
Gatilhos acionados automaticamente a cada Push ou Pull Request nas branches 'main' e 'develop'.
- ETAPAS DA ESTEIRA DE CI (GITHUB ACTIONS):
 1. Checkout & Setup: Instalação do ambiente Node.js (v20.x) e HashiCorp Terraform CLI.
 2. Dependency Installation: Instalação limpa via 'npm ci' a partir do lockfile.
 3. Quality & Linting: Verificação estática de código com 'npm run lint'.
 4. Automated Testing: Execução da suíte de testes unitários e de integração com 'npm test'.
 5. Build Verification: Compilação dos artefatos da aplicação via 'npm run build'.
 6. IaC Validation: Verificação de sintaxe e formatação do Terraform com 'terraform fmt' e 'terraform validate'.

c) **ESPECIFICAÇÃO DETALHADA DA INFRAESTRUTURA NECESSÁRIA:**

- REDE (VPC & SUBNETS):
 - 1 VPC isolada (CIDR 10.0.0.0/16) com Internet Gateway (IGW).
 - 2 Subnets Públicas (10.0.1.0/24 e 10.0.2.0/24) para a camada Web e Load Balancer.
 - 2 Subnets Privadas (10.0.10.0/24 e 10.0.11.0/24) para isolamento da camada de dados.
- APLICAÇÃO WEB & SEGURANÇA:
 - Security Group 'medqr-web-sg' liberando portas HTTP (80), HTTPS (443) e App (3000).
- BANCO DE DADOS GERENCIADO (RDS POSTGRESQL):
 - Instância AWS RDS PostgreSQL v15.4 (t3.micro) em subnets privadas.
 - Security Group 'medqr-db-sg' liberando a porta 5432 EXCLUSIVAMENTE para a camada Web.
- ARMAZENAMENTO ESTÁTICO (S3):
 - Bucket AWS S3 'medqr-app-assets-storage-2026' para hospedagem de assets e imagens de QR Code.

SEÇÃO 2

Pipeline de Integração Contínua (CI)

a) **CONFIGURAÇÃO DE UM REPOSITÓRIO DE CÓDIGO:**

Insira o link de acesso ao repositório (GitHub).

• LINK DO REPOSITÓRIO:

`https://github.com/samuelaiedo/medqr-clinical-system`

• ESTRUTURA DO REPOSITÓRIO:

- / (Código da Aplicação MedQR)
- /.github/workflows/ci.yml (Pipeline de CI do GitHub Actions)
- /devops/terraform/ (Scripts de Infraestrutura como Código)
 - ├─ main.tf (Provedor AWS e tags)
 - ├─ variables.tf (Parâmetros e credenciais)
 - ├─ vpc.tf (VPC, Subnets e IGW)
 - └─ resources.tf (Security Groups, RDS e S3)

- b) **IMPLEMENTAÇÃO DE UM PIPELINE DE CI USANDO UM GITHUB ACTIONS:**
Insira o link de acesso ao repositório (GitHub).
- EVIDÊNCIA DO ARQUIVO DE WORKFLOW (.github/workflows/ci.yml).

```
name: MedQR Continuous Integration Pipeline
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20.x', cache: 'npm' }
      - run: npm ci
      - run: npm run lint --if-present
      - run: npm test --if-present
      - run: npm run build --if-present

  terraform-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
      - run: terraform fmt -check -recursive devops/terraform
      - run: cd devops/terraform && terraform init -backend=false && terraform validate
```

SEÇÃO 3

Scripts de Infraestrutura como Código

a) **SCRIPTS PARA PROVISIONAMENTO DE INFRAESTRUTURA A PARTIR DA FERRAMENTA:**

Insira o link de acesso ao repositório (GitHub).

- FERRAMENTA UTILIZADA: HashiCorp Terraform (v1.6+)
- LINK DOS SCRIPTS NO REPOSITÓRIO:
<https://github.com/samuelaiedo/medqr-clinical-system/tree/main/devops/terraform>
- RESUMO DOS ARQUIVOS E RECURSOS:
 1. main.tf: Provider 'hashicorp/aws' (~> 5.0) com default tags 'Project=MedQR'.
 2. variables.tf: Configuração flexível de Região (us-east-1), CIDR (10.0.0.0/16) e BD.
 3. vpc.tf: Provisiona VPC, 2 Subnets Públicas, 2 Subnets Privadas e Internet Gateway.
 4. resources.tf: Provisiona Security Groups (Web e DB), Instância RDS PostgreSQL 15.4 e Bucket S3.
- VALIDAÇÃO:
 - 'terraform fmt -check': Executado com sucesso (0 desalinhamentos).
 - 'terraform validate': Configuração válida e testada com sucesso!